Name: Mohamad Khairul Bin Mansor

Report & Technical Documentation: Processing Utility Bills Using Box MCP Server

Introduction and Problem Statement

Each householder or small business owner typically receive recurring utility bills such as electricity, gas, water, waste, internet and phone bills from their utility providers in the form of PDF files. In order to track the spend, data and charges on each bill, the information need to be extracted into a spreadsheet or database manually. This process is tedious for a small handful of bills per month and let alone impossible across many providers, different account holders and larger time period.

The difficulty arises when the bills are semi-structured or unstructured documents. The information required is usually present in the bills, but its layout, wording and formatting are different from one provider to another. The information can also change over time, which is why naïve automation might fail in capturing them correctly.

This problem is considered an AI problem as it can't be solved robustly using deterministic rules alone as the rules would need to be rewritten for each new provider layout. It needs a model that can generalise over different layout and wording variation while still returning a precise and structured answer. Therefore, the information extraction problem is generally under sub-field of natural language processing within artificial intelligence.

Objectives

- Automatically view and read each utility bill in a cloud storage folder such as Box without hardcoding the files names and utility providers
- Extract relevant structured fields per bill without giving any template for each utility provider
- Aggregate the extracted information and provide answer to any question related to the utility bills using natural language
- Integrate the script with external data source such as Box and package the pipeline behind a demo application

Comparison of Approaches

Few approaches have been surveyed to solve the information-extraction problem. The simplest approach without any software engineering effort is manual data entry where the user reads each utility bill and record the values into a spreadsheet. This approach is accurate as long as it is done correctly. However, manual entry is slow, error-prone and does not scale nor automatable. Rule-based Optical Character Recognition (OCR) converts the PDF file into text format and uses fixed-position rules or regular expression to extract the fields. This OCR approach is fully deterministic and easy to run. The limitation is creation of a template per utility provider is required plus it is fragile to any layout change.

Classical machine learning model such as named entity recognition is another option as the statistical labelling model is trained to tag entities such as bill amount, date and name. It generalises better than regular expression but requires a human-labelled training dataset per entity. Another approach is general-purpose Large Language Model with zero or few shot learning. Extracted text is passed to LLM with natural-language instruction describing what fields to return. No fine-tuning is needed and the LLM can generalise well across different utility providers. If there is a change in the field, the prompt can be amended instead of retraining the whole model. Nonetheless, there is no native access to external data sources using this approach.

The best solution to tackle the problem is by using combination of LLM and agentic tool calling via Model Context Protocol (MCP). LLM is given with a small discoverable set of external

tools through MCP and it decides which tool to call before reasoning over the results. This combination allow flexibility in prompting together with live access to the actual utility bills. The utility bills can be stored in a cloud storage that compatible with MCP such as Box. This protocol also decouples the model from Box-specific integration code. For these reasons, combination of LLM and Box MCP are chosen over the other alternatives.

Model Context Protocol

Model Context Protocol was designed by Anthropic to tackle the problem with Large Language Model's context. MCP is a universal and an open standard for AI application or AI system to communicate with external tool or data source using one single interface. The goal of MCP is to solve issue regarding AI application interoperability, instead of creating huge technical and engineering burden to the software developers.

MCP was created to handle compatibility concern by preventing any dependency to specific vendors of LLM. Instead of specifying how LLM calls a certain tool, MCP allows a universal protocol between its components. Three main components in MCP architecture. They are the host, the client and the server. The host is the application or system that control the overall interaction. An example of MCP host is Integrated Development Environment (IDE) and custom AI agent. MCP client serves as a lightweight connector and maintain 1-to-1 session between host and server. The client converts natural language request from LLM into a structured JSON-RCP and translates the response back to the AI application. Lastly, MCP server provides specific data or capabilities such as databases, filesystems and API responses. Box MCP server provides tools that let AI agent interacts with Box content and features. Box's tools are callable functions that allow AI agent to perform certain actions such as executing a query.

Source Code Location

The complete file structure is as below:

```
├── README.md                    Project overview and setup instructions
├── app.py                       Streamlit demo application (UI)
├── invoice_pipeline.py          Core extraction pipeline (MCP + Gemini orchestration)
├── notebook.ipynb               Exploratory / prototyping notebook
├── requirements.txt             Python dependencies
├── env.example                  Template for required environment variables
├── .env                         Local runtime configuration (not shared)
├── invoices_example/            Sample utility bill PDFs used for validation
│   └── *.pdf                    Six monthly utility bill samples (Jan–Jun 2026)
└── mcp-server-box-main/         Vendored Box MCP Server
    ├── src/
    │   ├── mcp_server_box.py
    │   ├── server.py
    │   ├── config.py
    │   ├── tool_registry/
    │   └── tools/
    ├── docs/
    ├── tests/
    ├── pyproject.toml / uv.lock
    └── README.md / LICENSE
```

Architecture Overview

Streamlit UI browser is used as a front-end demo application that triggers utility bill extraction pipeline. It calls two external hosted services, Gemini API for LLM tool calling and a local Box MCP Server to connect to the utility bills stored in Box folder. A hosted Gemini endpoint of gemini-3.5-flash-lite is used instead of self-hosted model to remove CPU/GPU provisioning, model loading time and weight storage requirement.

Component Overview

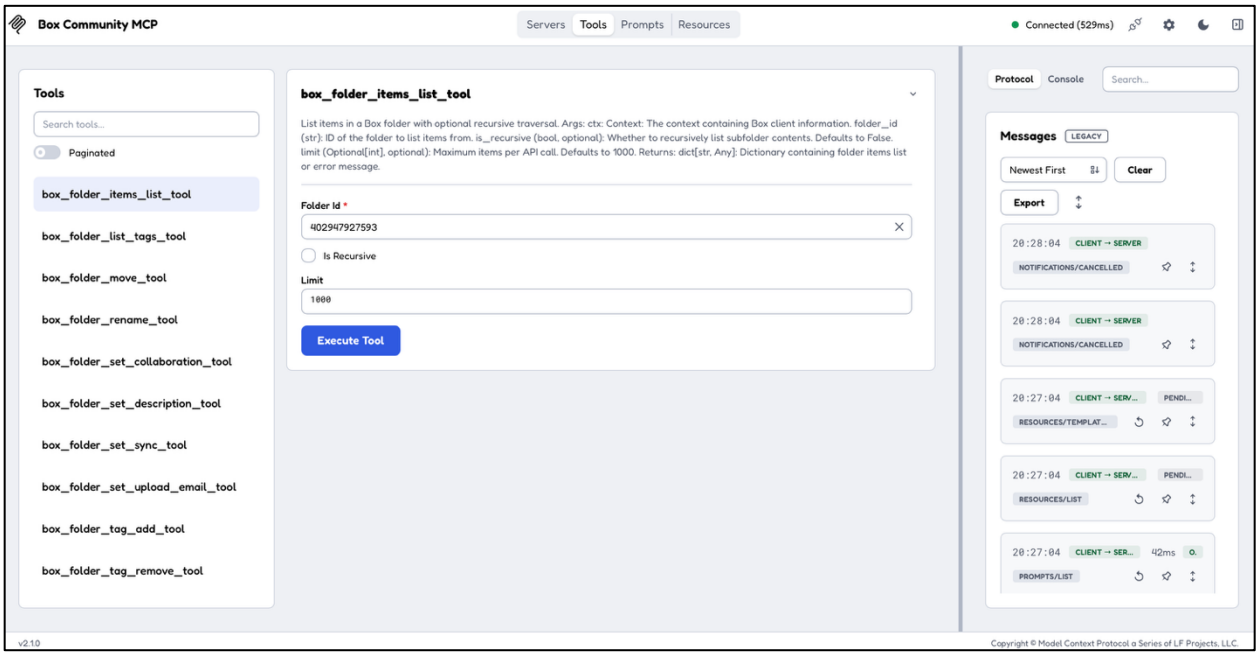
Component	Overview
app.py	Streamlit user interface that takes in Box folder ID as input. It triggers text extraction from the utility bills in the Box folder and produces result table and summary metrics. It also provides a free-text question box about the utility bills and generate answers based on the question
notebook.py	Jupyter notebook used for initial prototyping, developing and validating the pipeline against utility bills stored in Box folder

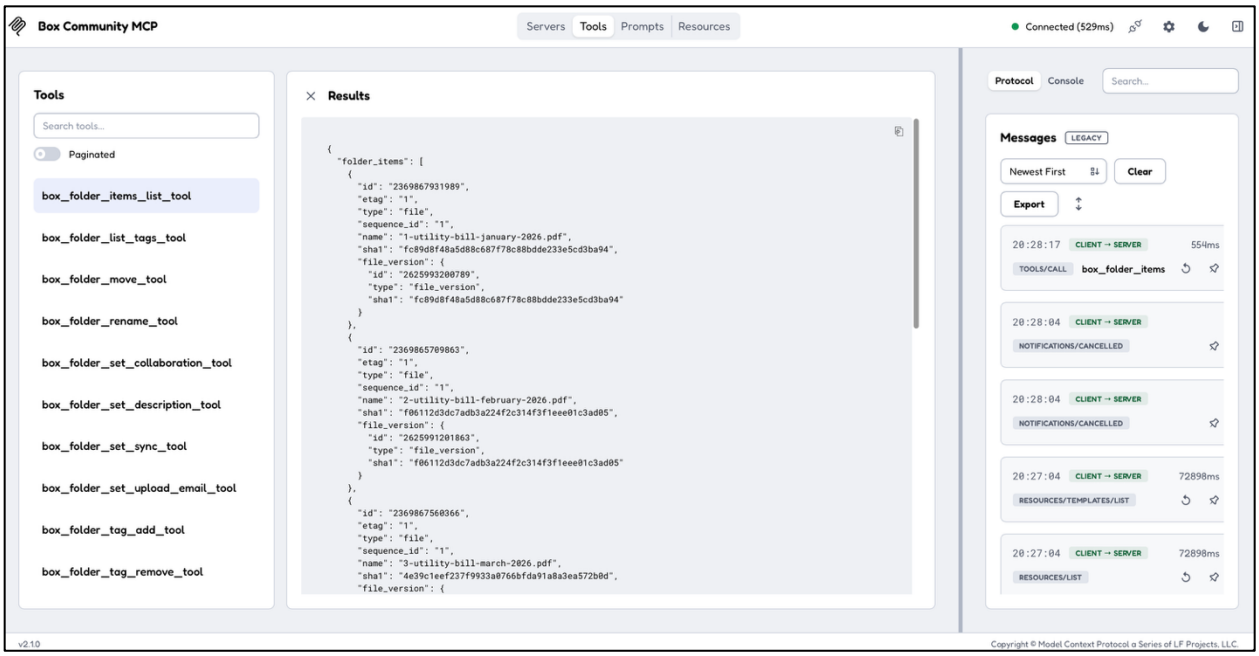
invoice_pipeline.py	The core pipeline containing functions to load environment configuration, connect to Box MCP Server, drive Gemini API tool calling, prompt Gemini to return structured JSON per bill and answer any question over the aggregated results
mcp-server-box-main folder	Box MCP Server local repository from https://github.com/box-community/mcp-server-box that exposes list of available tools. For reproducibility, mcp-server-box-main is uploaded as a zipped file
env.example	An example document that lists all required variables such as Box target folder ID, Box account CCG credentials and Gemini API key
invoices_example folder	Six sample of utility bills in PDF format generated synthetically to validate and test the pipeline end-to-end. The customer's name in the utility bills has been redacted for privacy consideration

Pipeline Flow

1. User opens Streamlit UI app and enters the Box folder ID containing the utility bills
2. invoice_pipeline.run_extraction() loads credentials from .env file and opens MCP ClientSession over a stdio subprocess running the Box MCP server
3. The pipeline connects gemini-3.5-flash-lite model with two whitelisted Box tools attached as function declarations. These tools are box_folder_items_list_tool and box_file_text_extract_tool
4. The pipeline asks the Gemini model to list the content of the folder ID and issues a function call, which is forwarded to Box MCP server using box_folder_items_list_tool and returns the folder's file listing
5. For each file in the list, the pipeline calls box_file_text_extract_tool directly to extract raw text from the utility bill
6. The raw text is embedded in a field-extraction prompt and sent to Gemini model which returns a JSON object with five target fields.
7. The results are accumulated into a list of dictionaries and transformed as pandas DataFrame with summary metrics and provider breakdown in the Streamlit UI.
8. Users ask a free-text question and ask_about_bills() sends the already-extracted JSON and the question to Gemini model. This returns its answer at the bottom of the page

MCP Inspector Web





MCP Inspector is an interactive developer tooling for testing and debugging MCP servers in the browser. When Box MCP server is connected, the list of all available tools within the server is displayed under Tools section. For example, when box_folder_items_list_tool is selected, the description of the tool is shown in the web and its input schema rendered as a

form. Filling in Folder ID of a Box folder and clicking Execute Tool button will display the results of the tool calling. In this case, all structured contents of the list of files inside the Box folder will be shown.

Configuration and Security

All credentials such as Box Client ID, Client Secret, Subject ID, Subject Type and Gemini API key are supplied via environment variables loaded from .env file. Box authentication uses Client Credentials Grant (CCG) which is a server-to-server OAuth flow suited for unattended backend service. Box MCP server exposes more than a hundred different tools but for this project, only two read-only tools are required and whitelisted. This limits both LLM's scope coverage and size of tool-definition payload sent on every Gemini model call. The Box MCP server runs as a local Standard Input/Output subprocess rather than an exposed network service. Hence, no additional network attack surface is introduced for the demo application.

Setting up the Pipeline

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Make sure `uv` is installed in order to run the Box MCP server
3. Upload the utility bills inside a Box folder
4. Copy `env.example` to `.env` and fill in the values:

```
cp env.example .env
```

- `BOX_FOLDER_ID` — ID of the Box folder containing the utility bills
- `BOX_CLIENT_ID`, `BOX_CLIENT_SECRET`, `BOX_SUBJECT_ID`, `BOX_SUBJECT_TYPE` — Box CCG app credentials
- `GEMINI_API_KEY` — Gemini API key

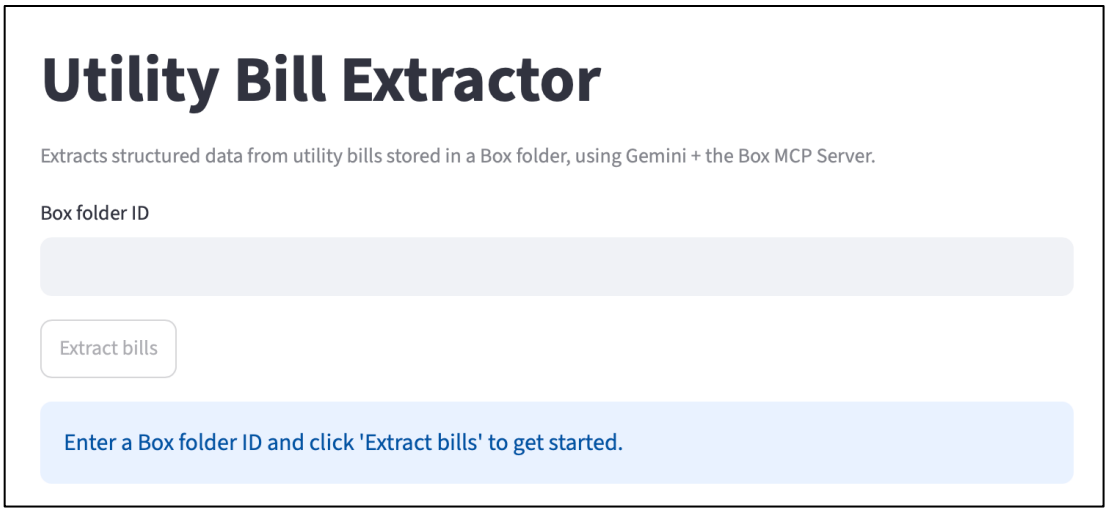
5. Make sure that the mcp-server-box-main directory is present alongside other files

Running the Pipeline

```
streamlit run app.py
```

- The Streamlit app will be automatically opened in the browser.
- Enter Box folder ID in the app, click **Extract bills** button and review the extracted data.
- Enter any question under Ask a question about these bills, click **Ask** button and review the generated answer.

Demo Application



Box folder ID

402947927593

Extract bills

Extracted bills

	customer_name	bill_amount_euro	provider_name	due_date	kWh_usage	file
0	Kxxx Mxxx	141.94	X Electric Company	21 February 2026	410	1-utility-bill-ja
1	Kxxx Mxxx	132.26	X Electric Company	21 March 2026	380	2-utility-bill-fe
2	Kxxx Mxxx	122.58	X Electric Company	21 April 2026	350	3-utility-bill-m
3	Kxxx Mxxx	106.45	X Electric Company	21 May 2026	300	4-utility-bill-a
4	Kxxx Mxxx	99.99	X Electric Company	21 June 2026	280	5-utility-bill-m
5	Kxxx Mxxx	106.45	X Electric Company	21 July 2026	300	6-utility-bill-ju

Summary

Total bills

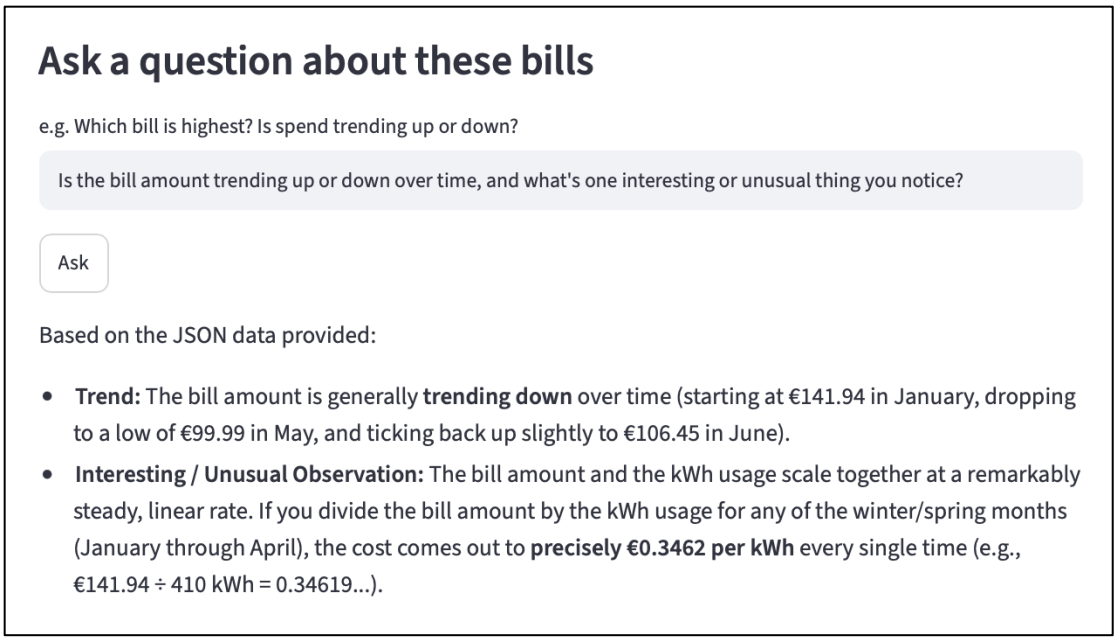
6

Total amount

€709.67

Breakdown by provider

provider_name	bills	total_amount
X Electric Company	6	709.67



The demo application is a single page Streamlit UI browser from app.py containing the following elements:

- Box folder ID: A text field to input the specified Box folder ID containing the utility bills
- Extract bill button: A click button that run extraction function with live progress status line
- Extracted bills: A table containing relevant information of all extracted bills from the Box folder
- Summary: Metrics showing total number of bills processed and total amount of the bills
- Breakdown by provider: A table displaying the bills grouped by provider_name
- Ask a question about these bills: A free-text field to allow user to ask any question related to the extracted data without any tool calling

The pipeline was tested end-to-end against a Box folder containing six sample utility bills. The Box MCP server correctly listed all six file names and extracted the texts for each bill. Gemini model of gemini-3.5-flash-lite accurately returned all five fields per bill. When user asked a question under Ask a question about these bills, the answer was displayed based on the JSON data provided. The extracted data is used for LLM reasoning such as deriving trend, relationship, interesting and unusual observation. The prompt can also be amended to tailor it to different bill types such as water or broadband bill so that relevant fields can be extracted.

Future Work

There are few limitations identified and future work can be done to improve the pipeline. These are:

- To add labelled evaluation and test set with ground truth labelling to measure extraction accuracy
- To apply the utility bills from variety of utility providers to test generalisation across different layouts
- To add error handling message if error occurred during extraction process
- To include the pipeline and Box MCP server inside Docker container for a consistent and isolated deployment
- To allow persistent storage of extraction results for example to a database
- To detect and redact any Personally Identifiable Information (PII) if real utility bill is used

Conclusion

This project set out to automate a tedious task of extracting utility bill data using an approach that generalises well across utility provider layout without hardcoding specific parsing rule per provider. After looking at various approaches from a software engineering perspective, LLM connected to its data source via Box MCP was selected and implemented as a working Streamlit demo application. It is validated end-to-end against sample of utility bills and packaged behind a deployment pipeline.

References

Anthropic. (2024). Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol

Box Community. mcp-server-box [GitHub repository]. https://github.com/box-community/mcp-server-box

Box Developer. Box developer documentation. https://developer.box.com

Box Developer. Box MCP guide. https://developer.box.com/guides/box-mcp

Google AI for Developers. Gemini API documentation. https://ai.google.dev/gemini-api/docs

Model Context Protocol. (2026). Getting started: Introduction. https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro